````
# Energy Sector PRO — CEG / VST / CCJ Quant Research

A reproducible equity-research pipeline for a 3-stock energy sleeve
(Constellation Energy, Vistra, Cameco), benchmarked against XLE, with
factor analysis, mean-variance portfolio optimization, and a Monte Carlo
simulation — delivered as both a professional Excel workbook and the
Python source that generates its non-formula-driven components.

## What's here

```
energy_report_pro/
├── README.md                ← this file
├── data/
│   └── market_data.csv      ← single source of truth: monthly prices/levels
│                              for CEG, VST, CCJ, XLE, WTI, Henry Hub,
│                              10Y Treasury yield, uranium spot (Jan-2025–Sep-2026)
├── src/
│   ├── fetch_data.py         ← data access layer (live-fetch stub + CSV fallback)
│   ├── monte_carlo.py        ← correlated multi-asset Monte Carlo simulation
│   └── build_workbook.py     ← builds the full .xlsx from data/ + output/*.csv
└── output/
    ├── Energy_Sector_PRO_Research.xlsx   ← the deliverable workbook
    ├── monte_carlo_summary.csv
    ├── monte_carlo_fan.csv
    └── monte_carlo_histogram.csv
```

## Reproducing the workbook from scratch

```bash
cd src
python3 monte_carlo.py       # writes output/monte_carlo_*.csv
python3 build_workbook.py    # reads data/ + output/*.csv, writes the .xlsx
```

Changing `data/market_data.csv` and re-running both scripts regenerates
every statistic and chart in the workbook — nothing is hand-typed into
the spreadsheet itself except the deep-dive qualitative text (fundamentals,
catalysts, bull/bear case) and the macro-factor sourcing notes.

## What the workbook contains

| Tab | Content |
|---|---|
| Cover | Scope, what's new, workbook map, methodology & limitations |
| Market Data | Monthly prices + XLE benchmark + 4 macro factors, sourced |
| Returns & Risk | Live-formula returns, real running-peak max drawdown, Sharpe, Sortino, historical VaR/CVaR 95%/99%, skew, kurtosis, correlation matrix |
| Benchmark & Factors | Alpha/beta/tracking-error/information-ratio/up-down-capture vs. XLE; univariate betas to oil, gas, uranium, rates |
| Portfolio Optimization | 66-point mean-variance efficient frontier (10% weight grid), max-Sharpe and min-variance portfolios identified via INDEX/MATCH, efficient-frontier chart |
| Monte Carlo | 10,000-path, 12-month simulation: percentile fan chart, terminal-return histogram, simulation-based VaR/CVaR |
| CEG / VST / CCJ Deep Dive | Fundamentals, technicals, earnings history, catalysts, bull/bear case |
| Catalyst Calendar | Consolidated dated events through Q1 2027 |
| Recommendation | Optimizer output + Monte Carlo scenario framing + sizing rationale |

## Methodology notes & honest limitations

- **Sample size.** 21 monthly observations (20 returns) is thin for
  estimating covariance, VaR, skew/kurtosis, and regression betas. Every
  number here should be read as directional, not precise.
- **Max drawdown** now uses the real running-peak-to-trough method
  (a `RunMax`/`Drawdown` helper table on the Returns & Risk tab), not a
  naive `MIN(price)/MAX(price)-1` shortcut — that shortcut can badly
  overstate drawdown when a series' low predates its high (this happened
  for CCJ: naive proxy said -67%, the real method says -30%, both are in
  the workbook's git history of this project for comparison).
- **Factor betas are univariate** (one macro factor regressed at a time).
  Oil, gas, uranium and rates are themselves correlated, so a proper
  multi-factor attribution would need to control for that collinearity —
  a job for `statsmodels`/`sklearn`, not single-variable `SLOPE()` calls.
  Treat Table B on the Benchmark & Factors tab as directional sensitivity,
  not clean independent exposures.
- **The Monte Carlo simulation is parametric** (multivariate normal fit
  to the 20-month sample), not a historical bootstrap. It does not
  capture fat tails, regime shifts (e.g. a PJM regulatory intervention),
  or return autocorrelation. It is a shape-of-uncertainty illustration.
- **WTI crude and uranium spot monthly series are analyst-reconstructed
````

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
  approximations**, not pulled from a live feed — the sandboxed build
  environment this was produced in could not reach EIA/FRED's raw data
  endpoints or a uranium-price vendor. XLE, Henry Hub, and the 10Y yield
  are sourced from public data providers. Replace `data/market_data.csv`
  with a Bloomberg/FactSet/EIA live pull before using the oil or uranium
  factor numbers for anything beyond illustration — `fetch_data.py` is
  structured so that swapping in a real data vendor requires no changes
  anywhere else in the pipeline.
- **This is not investment advice.** See the Cover and Recommendation
  tabs for the full disclaimer.
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```python
#!/usr/bin/env python3
"""
fetch_data.py
=============
Data access layer for the Energy Sector PRO research workbook.

Design intent (reproducibility):
    This module is the single place that knows how to get market data into
    the pipeline. In a live/production environment (with outbound internet
    access to market-data vendors), `fetch_live()` would pull fresh data via
    an API such as yfinance, Alpha Vantage, or a broker/Bloomberg feed. In
    this sandboxed build environment (no outbound access to those vendors),
    `fetch_live()` falls back automatically to the static, source-documented
    snapshot in data/market_data.csv — the same file the Excel workbook is
    built from, so the spreadsheet and the Python analysis never disagree.

    Swap in real credentials/an API key and `fetch_live()` starts pulling
    live data with zero changes needed elsewhere in the pipeline — that's
    the point of isolating I/O behind one function.

Usage:
    from fetch_data import load_market_data
    df = load_market_data()
"""
from __future__ import annotations
import os
import pandas as pd

HERE = os.path.dirname(os.path.abspath(__file__))
CSV_PATH = os.path.join(HERE, "..", "data", "market_data.csv")

TICKERS = ["CEG", "VST", "CCJ"]
BENCHMARK = "XLE"
FACTORS = ["WTI", "HENRY_HUB", "UST10Y", "URANIUM"]


def fetch_live(tickers=None, start="2025-01-01", end="2026-09-30"):
    """
    Attempt to pull live monthly data via yfinance. Falls back to the
    static CSV snapshot if yfinance / network access is unavailable
    (e.g. this sandboxed build environment has no route to Yahoo Finance).
    """
    tickers = tickers or TICKERS + [BENCHMARK]
    try:
        import yfinance as yf  # noqa: F401
        frames = {}
        for t in tickers:
            hist = yf.download(t, start=start, end=end, interval="1mo", progress=False)
            frames[t] = hist["Close"]
        df = pd.DataFrame(frames)
        df.index.name = "date"
        print("[fetch_data] Pulled live data via yfinance.")
        return df
    except Exception as exc:  # noqa: BLE001
        print(f"[fetch_data] Live fetch unavailable ({exc!r}); "
              f"falling back to data/market_data.csv snapshot.")
        return load_market_data()


def load_market_data() -> pd.DataFrame:
    """Load the static, source-documented monthly snapshot used by both
    the Excel workbook and the Monte Carlo simulation. See README.md for
    the exact source for each column (CEG/VST/CCJ/XLE price history and
    WTI/Henry Hub/UST10Y/Uranium factor series)."""
    df = pd.read_csv(CSV_PATH)
    return df


def monthly_returns(df: pd.DataFrame, cols=None) -> pd.DataFrame:
    """Simple percentage monthly returns for price/level columns."""
    cols = cols or (TICKERS + [BENCHMARK])
    return df[cols].pct_change().dropna().reset_index(drop=True)


if __name__ == "__main__":
    data = load_market_data()
    print(data.head())
    print("\nMonthly returns (CEG/VST/CCJ/XLE):")
    print(monthly_returns(data).head())
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```python
#!/usr/bin/env python3
"""
monte_carlo.py
==============
Correlated multi-asset Monte Carlo simulation for the CEG / VST / CCJ
3-stock energy sleeve, used to generate a 12-month forward distribution
of outcomes instead of the single eyeballed bull/base/bear price a junior
analyst would hand-wave.

Method
------
1. Estimate the monthly mean-return vector (mu) and covariance matrix
   (Sigma) from the trailing 20 monthly observations (Jan-2025–Sep-2026).
2. Cholesky-decompose Sigma to generate correlated normal shocks.
3. Simulate N independent 12-month paths for each of the 3 stocks,
   compound them monthly, and roll them up into a fixed-weight portfolio
   (default: 40% CEG / 35% VST / 25% CCJ, matching the Recommendation tab).
4. Report the terminal (12-month) distribution: percentiles, 95%/99%
   simulation-based VaR and CVaR, and probability of loss.
5. Save path data and summary statistics to output/ as CSV so the Excel
   workbook can embed the results as a fan chart + histogram without
   needing a live Python kernel.

Caveats (stated explicitly, as a senior risk analyst would):
    - Returns are modeled as i.i.d. multivariate normal draws from a
      20-observation monthly sample. 20 data points is a thin sample for
      estimating mean/covariance; the simulation is a useful illustration
      of *shape* of uncertainty, not a precise forecast. Fat tails, regime
      shifts (e.g. a PJM regulatory intervention), and autocorrelation in
      energy-sector returns are NOT captured by this simple model.
    - This is a parametric (normal) Monte Carlo, not a bootstrap of actual
      historical return sequences. A bootstrap/historical-sampling variant
      would better preserve the real fat-tailed, skewed shape of monthly
      returns and is a natural next iteration.

Usage:
    python3 monte_carlo.py
"""
from __future__ import annotations
import os
import numpy as np
import pandas as pd

from fetch_data import load_market_data, monthly_returns, TICKERS

HERE = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(HERE, "..", "output")
os.makedirs(OUT_DIR, exist_ok=True)

N_SIMS = 10_000
N_MONTHS = 12
WEIGHTS = {"CEG": 0.40, "VST": 0.35, "CCJ": 0.25}
RISK_FREE_ANNUAL = 0.043
SEED = 42


def run_simulation(n_sims=N_SIMS, n_months=N_MONTHS, weights=None, seed=SEED):
    weights = weights or WEIGHTS
    df = load_market_data()
    rets = monthly_returns(df, cols=TICKERS)

    mu = rets.mean().values                    # (3,) monthly mean return vector
    sigma = rets.cov().values                  # (3,3) monthly covariance matrix (sample, ddof=1 de
fault)

    rng = np.random.default_rng(seed)
    L = np.linalg.cholesky(sigma)

    # correlated standard normal shocks -> correlated monthly returns
    z = rng.standard_normal(size=(n_sims, n_months, 3))
    corr_shocks = z @ L.T                          # (n_sims, n_months, 3)
    sim_returns = mu[None, None, :] + corr_shocks  # (n_sims, n_months, 3)

    # compound each asset's path to a cumulative index (start = 1.0)
    cum_index = np.cumprod(1 + sim_returns, axis=1)  # (n_sims, n_months, 3)

    w = np.array([weights[t] for t in TICKERS])
    # portfolio index path = weighted sum of each asset's compounded index
    port_path = cum_index @ w                        # (n_sims, n_months)

    terminal = port_path[:, -1]                      # (n_sims,) terminal 12-mo portfolio multiple
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
81
82      # percentile fan across the 12-month horizon (for charting)
83      pctiles = [5, 10, 25, 50, 75, 90, 95]
84      fan = np.percentile(port_path, pctiles, axis=0)   # (len(pctiles), n_months)
85      fan_df = pd.DataFrame(fan.T, columns=[f"p{p}" for p in pctiles])
86      fan_df.insert(0, "month", np.arange(1, n_months + 1))
87      fan_df.to_csv(os.path.join(OUT_DIR, "monte_carlo_fan.csv"), index=False)
88
89      terminal_returns = terminal - 1.0  # express as a return, not a multiple
90      var95 = np.percentile(terminal_returns, 5)
91      var99 = np.percentile(terminal_returns, 1)
92      cvar95 = terminal_returns[terminal_returns <= var95].mean()
93      cvar99 = terminal_returns[terminal_returns <= var99].mean()
94      prob_loss = float((terminal_returns < 0).mean())
95      prob_loss_10 = float((terminal_returns < -0.10).mean())
96      prob_gain_25 = float((terminal_returns > 0.25).mean())
97
98      summary = {
99          "n_sims": n_sims,
100         "horizon_months": n_months,
101         "weights": weights,
102         "mean_terminal_return": float(terminal_returns.mean()),
103         "median_terminal_return": float(np.median(terminal_returns)),
104         "stdev_terminal_return": float(terminal_returns.std(ddof=1)),
105         "var_95": float(var95),
106         "var_99": float(var99),
107         "cvar_95": float(cvar95),
108         "cvar_99": float(cvar99),
109         "prob_loss": prob_loss,
110         "prob_loss_gt_10pct": prob_loss_10,
111         "prob_gain_gt_25pct": prob_gain_25,
112         "p5_terminal_return": float(np.percentile(terminal_returns, 5)),
113         "p25_terminal_return": float(np.percentile(terminal_returns, 25)),
114         "p50_terminal_return": float(np.percentile(terminal_returns, 50)),
115         "p75_terminal_return": float(np.percentile(terminal_returns, 75)),
116         "p95_terminal_return": float(np.percentile(terminal_returns, 95)),
117     }
118     summary_df = pd.DataFrame([summary])
119     summary_df.to_csv(os.path.join(OUT_DIR, "monte_carlo_summary.csv"), index=False)
120
121     # histogram bins of terminal returns for charting
122     counts, bin_edges = np.histogram(terminal_returns, bins=30)
123     hist_df = pd.DataFrame({
124         "bin_left": bin_edges[:-1],
125         "bin_right": bin_edges[1:],
126         "count": counts,
127         "freq": counts / n_sims,
128     })
129     hist_df.to_csv(os.path.join(OUT_DIR, "monte_carlo_histogram.csv"), index=False)
130
131     return summary, fan_df, hist_df
132
133
134 if __name__ == "__main__":
135     summary, fan_df, hist_df = run_simulation()
136     print("Monte Carlo summary (12-month portfolio, 40/35/25 CEG/VST/CCJ):")
137     for k, v in summary.items():
138         print(f"  {k}: {v}")
139     print(f"\nWrote: {OUT_DIR}/monte_carlo_summary.csv")
140     print(f"Wrote: {OUT_DIR}/monte_carlo_fan.csv")
141     print(f"Wrote: {OUT_DIR}/monte_carlo_histogram.csv")
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```python
#!/usr/bin/env python3
"""
build_workbook.py
==================
Builds the senior-level "Energy Sector PRO" equity research workbook:
CEG / VST / CCJ vs. the XLE benchmark, with live Excel formulas for
returns, risk, factor exposure, mean-variance optimization, plus embedded
(pre-computed, clearly labeled) Monte Carlo simulation output.

Run from this directory:  python3 build_workbook.py
Then recalc + verify with the xlsx skill's scripts/recalc.py before
treating the output as final.
"""
import os
import csv
import ast
import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter
from openpyxl.chart import LineChart, BarChart, ScatterChart, Reference, Series
from openpyxl.chart.marker import Marker

from fetch_data import load_market_data, TICKERS, BENCHMARK, FACTORS

HERE = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(HERE, "..", "output")
OUT_PATH = os.path.join(OUT_DIR, "Energy_Sector_PRO_Research.xlsx")

# ---------------------------------------------------------------- styling --
FONT = "Arial"
NAVY = "1F2A44"
STEEL = "2E5A88"
LIGHT_BLUE = "DCE6F1"
LIGHT_GRAY = "F2F2F2"
GOLD = "C9A646"
GREEN_FILL = "E2EFDA"
RED_FILL = "FCE4E4"
YELLOW_FILL = "FFF2CC"
WHITE = "FFFFFF"
BLACK = "000000"
BLUE_INPUT = "0000FF"
GREEN_LINK = "008000"


def style_title(cell, size=16, color=WHITE, bold=True):
    cell.font = Font(name=FONT, size=size, bold=bold, color=color)


def style_header(cell, bg=NAVY, color=WHITE, size=10, bold=True):
    cell.font = Font(name=FONT, size=size, bold=bold, color=color)
    cell.fill = PatternFill("solid", fgColor=bg)
    cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)


def style_label(cell, bold=True, size=10, color=BLACK, italic=False):
    cell.font = Font(name=FONT, size=size, bold=bold, color=color, italic=italic)


def style_body(cell, size=10, color=BLACK, bold=False, italic=False, input_cell=False, link_cell=False
):
    c = color
    if input_cell:
        c = BLUE_INPUT
    elif link_cell:
        c = GREEN_LINK
    cell.font = Font(name=FONT, size=size, color=c, bold=bold, italic=italic)


def thin_border():
    s = Side(style="thin", color="BFBFBF")
    return Border(left=s, right=s, top=s, bottom=s)


def box(ws, cell_range, bg=None, border=True):
    for row in ws[cell_range]:
        for c in row:
            if bg:
                c.fill = PatternFill("solid", fgColor=bg)
            if border:
                c.border = thin_border()

```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
81
82 def set_col_widths(ws, widths):
83     for i, w in enumerate(widths, start=1):
84         ws.column_dimensions[get_column_letter(i)].width = w
85
86
87 def new_sheet(wb, name):
88     if name in wb.sheetnames:
89         del wb[name]
90     ws = wb.create_sheet(name)
91     ws.sheet_view.showGridLines = False
92     return ws
93
94
95 def read_csv_dict(path):
96     with open(path, newline="") as f:
97         return list(csv.DictReader(f))
98
99
100 MONTHS = None  # populated in build_market_data
101
102
103 # ============================================================================
104 # 1. COVER
105 # ============================================================================
106 def build_cover(wb):
107     cover = wb["Sheet"] if "Sheet" in wb.sheetnames and len(wb.sheetnames) == 1 else new_sheet(wb, "Co
    ver")
108     if cover.title != "Cover":
109         cover.title = "Cover"
110     set_col_widths(cover, [4, 28, 28, 28, 28, 4])
111     for r in range(1, 46):
112         cover.row_dimensions[r].height = 18
113
114     box(cover, "A1:F12", bg=NAVY, border=False)
115     cover.merge_cells("B2:E2")
116     cover["B2"] = "ENERGY SECTOR — SENIOR QUANT RESEARCH"
117     style_title(cover["B2"], size=20, color=WHITE)
118     cover.merge_cells("B3:E3")
119     cover["B3"] = "CEG · VST · CCJ — Benchmark, Factor, Optimization & Monte-Carlo Framework"
120     style_title(cover["B3"], size=13, color=GOLD, bold=False)
121
122     cover.merge_cells("B5:E5")
123     cover["B5"] = "Independent Power Producers & Nuclear Fuel — AI/Data-Center Power Demand Thematic"
124     style_body(cover["B5"], size=11, color=LIGHT_BLUE, italic=True)
125
126     meta = [
127         ("Report date:", "2026-10-01"),
128         ("Coverage window:", "Jan-2025 — Sep-2026 (trailing), fwd view to FY2027"),
129         ("Prepared for:", "xristos.katsambas@gmail.com"),
130         ("Classification:", "Internal research workbook — not a solicitation"),
131     ]
132     row = 7
133     for label, val in meta:
134         cover.merge_cells(f"B{row}:C{row}")
135         cover[f"B{row}"] = label
136         style_body(cover[f"B{row}"], color=WHITE, bold=True)
137         cover[f"D{row}"] = val
138         style_body(cover[f"D{row}"], color=WHITE)
139         row += 1
140
141     row = 14
142     cover.merge_cells(f"B{row}:E{row}")
143     cover[f"B{row}"] = "WHAT'S NEW IN THIS VERSION (vs. the original workbook)"
144     style_header(cover[f"B{row}"], bg=STEEL)
145     row += 1
146     upgrades = [
147         "XLE sector benchmark added — alpha, tracking error, information ratio, up/down capture",
148         "Macro factor sensitivities — single-factor betas to WTI crude, Henry Hub gas, uranium spot, 1
    0Y yield",
149         "Real running-peak max drawdown (replaces the MIN/MAX proxy) + historical VaR/CVaR (95%/99%) +
     skew/kurtosis",
150         "Sortino ratio (downside deviation) alongside Sharpe",
151         "Mean-variance portfolio optimization — 66-point efficient frontier, max-Sharpe & min-vol port
    folios identified",
152         "Monte Carlo simulation (10,000 paths, 12-month horizon) — percentile fan chart, terminal-valu
    e histogram, simulation-based VaR/CVaR",
153         "Companion Python repo (src/) for full reproducibility: fetch_data.py, monte_carlo.py, build_w
    orkbook.py",
154     ]
155     for u in upgrades:
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
        cover.merge_cells(f"B{row}:E{row}")
        cover.cell(row=row, column=2, value=f"✓  {u}")
        style_body(cover.cell(row=row, column=2), size=9.5)
        cover.cell(row=row, column=2).fill = PatternFill("solid", fgColor=GREEN_FILL)
        cover.row_dimensions[row].height = 20
        row += 1

    row += 1
    cover.merge_cells(f"B{row}:E{row}")
    cover[f"B{row}"] = "WORKBOOK MAP"
    style_header(cover[f"B{row}"], bg=STEEL)
    row += 1
    tabs = [
        ("1. Cover", "This page — scope, what's new, disclaimers"),
        ("2. Market Data", "Monthly levels: CEG/VST/CCJ/XLE + WTI, Henry Hub, 10Y yield, uranium spot — sourced"),
        ("3. Returns & Risk", "Returns, real drawdown, Sharpe/Sortino, VaR/CVaR, skew/kurtosis — live formulas"),
        ("4. Benchmark & Factors", "Alpha/beta/TE/IR/capture vs. XLE; univariate betas to oil/gas/uranium/rates"),
        ("5. Portfolio Optimization", "66-point efficient frontier, max-Sharpe & min-variance portfolios, CML"),
        ("6. Monte Carlo", "10,000-path simulation: percentile fan, terminal histogram, sim VaR/CVaR"),
        ("7-9. Deep Dives", "CEG / VST / CCJ — fundamentals, technicals, catalysts, earnings history"),
        ("10. Catalyst Calendar", "Consolidated dated events, Oct-2026–Q1-2027"),
        ("11. Recommendation", "Optimizer + simulation-informed sizing, scenario framing, disclaimer"),
    ]
    for name, desc in tabs:
        cover.cell(row=row, column=2, value=name)
        style_label(cover.cell(row=row, column=2), bold=True, size=9.5)
        cover.merge_cells(f"C{row}:E{row}")
        cover.cell(row=row, column=3, value=desc)
        style_body(cover.cell(row=row, column=3), size=9.5)
        row += 1
    box(cover, f"B{row-len(tabs)}:E{row-1}", border=True)

    row += 2
    cover.merge_cells(f"B{row}:E{row+5}")
    cover[f"B{row}"] = (
        "METHODOLOGY & LIMITATIONS (read before using any number in this workbook): Monthly observation "
        "frequency (21 months) is a THIN sample for statistical inference — covariance, VaR, skew/kurtosis and "
        "regression betas all carry wide estimation error at n=20. Max drawdown, VaR/CVaR and correlation use "
        "month-end data only and will UNDERSTATE true intra-month peak-to-trough risk. Macro factor betas are "
        "univariate (one factor at a time) — true multi-factor attribution requires controlling for collinearity "
        "among oil/gas/uranium/rates, which is a job for proper regression software, not single-variable Excel "
        "formulas; treat these as directional, not precise. The Monte Carlo simulation assumes i.i.d. multivariate-"
        "normal monthly returns — it does not capture fat tails, regime shifts, or autocorrelation, and is a "
        "shape-of-uncertainty illustration, not a forecast. WTI and uranium monthly series are analyst-"
        "reconstructed approximations (see Market Data sourcing note); XLE, Henry Hub, and 10Y yield are sourced "
        "from public data providers. This is NOT investment advice or a broker-dealer research report. Verify "
        "every figure against a live terminal before acting."
    )
    cover[f"B{row}"].alignment = Alignment(wrap_text=True, vertical="top")
    style_body(cover[f"B{row}"], size=8.5, italic=True, color="595959")
    return cover


# ============================================================================
# 2. MARKET DATA
# ============================================================================
def build_market_data(wb):
    global MONTHS
    md = new_sheet(wb, "Market Data")
    set_col_widths(md, [12, 11, 11, 11, 11, 10, 11, 10, 11])
    cols = ["Month", "CEG", "VST", "CCJ", "XLE", "WTI", "Henry Hub", "UST 10Y", "Uranium"]

    md.merge_cells("A1:I1")
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    md["A1"] = "MONTHLY MARKET DATA — Prices, Benchmark & Macro Factors (Jan-2025 — Sep-2026)"
    style_header(md["A1"], size=13)
    md.row_dimensions[1].height = 22

    for i, h in enumerate(cols):
        c = md.cell(row=2, column=1 + i, value=h)
        style_header(c, bg=STEEL)

    data_path = os.path.join(HERE, "..", "data", "market_data.csv")
    rows_data = read_csv_dict(data_path)
    MONTHS = [r["month"] for r in rows_data]

    start_row = 3
    for i, r in enumerate(rows_data):
        rr = start_row + i
        md.cell(row=rr, column=1, value=r["month"])
        style_label(md.cell(row=rr, column=1), bold=False, size=9.5)
        vals = [float(r["CEG"]), float(r["VST"]), float(r["CCJ"]), float(r["XLE"]),
                float(r["WTI"]), float(r["HENRY_HUB"]), float(r["UST10Y"]), float(r["URANIUM"])]
        for ci, v in enumerate(vals):
            cell = md.cell(row=rr, column=2 + ci, value=v)
            cell.number_format = "$#,##0.00" if ci < 4 else ("0.00%" if False else "#,##0.00")
            style_body(cell, size=9.5, input_cell=True)
            cell.alignment = Alignment(horizontal="center")
        box(md, f"A{rr}:I{rr}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    last_row = start_row + len(rows_data) - 1
    box(md, "A2:I2")

    spot_row = last_row + 3
    md.cell(row=spot_row, column=1, value="Spot (2026-10-01, intraday)")
    style_label(md.cell(row=spot_row, column=1))
    spot_vals = [264.58, 139.05, 87.39, 61.80, 62.00, 2.85, 4.98, 89.45]
    for ci, v in enumerate(spot_vals):
        cell = md.cell(row=spot_row, column=2 + ci, value=v)
        cell.number_format = "#,##0.00"
        style_body(cell, size=9.5, bold=True, input_cell=True)
        cell.alignment = Alignment(horizontal="center")
        cell.fill = PatternFill("solid", fgColor=LIGHT_BLUE)
    box(md, f"A{spot_row}:I{spot_row}")

    note_row = spot_row + 2
    md.merge_cells(f"A{note_row}:I{note_row}")
    md.cell(row=note_row, column=1, value=(
        "Sources: CEG/VST/CCJ — digrin.com, StockAnalysis.com, Yahoo Finance, Robinhood (retrieved 202
6-10-01). "
        "XLE — digrin.com monthly closes. Henry Hub natural gas spot ($/MMBtu) — U.S. EIA (eia.gov/dna
v/ng), "
        "Sep-2026 value estimated (not yet published at report date). UST 10Y — multpl.com monthly ave
rages, "
        "cross-checked against market commentary for Sep-2026. Uranium spot (U3O8, $/lb) — Sep-2026 va
lue sourced "
        "from Trading Economics; Jan-2025–Aug-2026 are ANALYST-RECONSTRUCTED APPROXIMATIONS consistent
 with the "
        "publicly reported direction of the uranium market (IMF/NUEXCO series not independently reprod
ucible here) "
        "— replace with a UxC/TradeTech subscription feed before relying on these for the Uranium fact
or column. "
        "WTI crude ($/bbl) is similarly an ANALYST-RECONSTRUCTED APPROXIMATION based on EIA STEO comme
ntary "
        "(direct EIA/FRED time-series endpoints were not reachable from this environment) — replace wi
th a live "
        "feed (EIA, Bloomberg, FactSet) before using the Oil factor column for anything beyond directi
onal illustration."
    ))
    style_body(md.cell(row=note_row, column=1), size=8, color="7F7F7F", italic=True)
    md.cell(row=note_row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
    md.row_dimensions[note_row].height = 80
    md.freeze_panes = "A3"

    return {"start_row": start_row, "last_row": last_row, "spot_row": spot_row,
            "cols": {"Month": "A", "CEG": "B", "VST": "C", "CCJ": "D", "XLE": "E",
                     "WTI": "F", "HH": "G", "UST10Y": "H", "URANIUM": "I"}}


# ============================================================================
# 3. RETURNS & RISK
# ============================================================================
def build_returns_risk(wb, md_rows):
    rr = new_sheet(wb, "Returns & Risk")
    set_col_widths(rr, [11, 10, 10, 10, 10, 3, 11, 11, 10, 11])
    mdc = md_rows["cols"]
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    md_sr, md_lr = md_rows["start_row"], md_rows["last_row"]

    rr.merge_cells("A1:J1")
    rr["A1"] = "RETURNS & RISK METRICS — live formulas, linked to Market Data"
    style_header(rr["A1"], size=13)
    rr.row_dimensions[1].height = 22

    headers = ["Month", "CEG", "VST", "CCJ", "XLE", "", "WTI %chg", "HH %chg", "UST10Y Δ", "Uranium %chg"]
    for i, h in enumerate(headers):
        if h == "":
            continue
        c = rr.cell(row=2, column=1 + i, value=h)
        style_header(c, bg=STEEL)

    ret_first, ret_last = md_sr + 1, md_lr  # 4..23
    for i, md_r in enumerate(range(md_sr + 1, md_lr + 1)):
        rrow = ret_first + i
        rr.cell(row=rrow, column=1, value=f"='Market Data'!A{md_r}")
        style_label(rr.cell(row=rrow, column=1), bold=False, size=9)
        prev = md_r - 1
        # price/level return columns: CEG,VST,CCJ,XLE
        for ci, key in zip([2, 3, 4, 5], ["CEG", "VST", "CCJ", "XLE"]):
            col = mdc[key]
            f = f"=('Market Data'!{col}{md_r}/'Market Data'!{col}{prev})-1"
            cell = rr.cell(row=rrow, column=ci, value=f)
            cell.number_format = "0.00%"
            style_body(cell, size=9)
            cell.alignment = Alignment(horizontal="center")
        # factor changes: WTI%,HH%,UST10Y(level diff),Uranium%
        for ci, key, pct in zip([7, 8, 9, 10], ["WTI", "HH", "UST10Y", "URANIUM"], [True, True, False, True]):
            col = mdc[key]
            if pct:
                f = f"=('Market Data'!{col}{md_r}/'Market Data'!{col}{prev})-1"
            else:
                f = f"=('Market Data'!{col}{md_r}-'Market Data'!{col}{prev})"
            cell = rr.cell(row=rrow, column=ci, value=f)
            cell.number_format = "0.00%" if pct else "0.00"
            style_body(cell, size=9)
            cell.alignment = Alignment(horizontal="center")
        box(rr, f"A{rrow}:J{rrow}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    box(rr, "A2:J2")

    # ---- Downside-squared-deviation helper columns (for Sortino), cols L,M,N ----
    rr.cell(row=2, column=12, value="CEG dwn²")
    rr.cell(row=2, column=13, value="VST dwn²")
    rr.cell(row=2, column=14, value="CCJ dwn²")
    for c in (12, 13, 14):
        style_header(rr.cell(row=2, column=c), bg="8EA9C1", size=8.5)
    for i, rrow in enumerate(range(ret_first, ret_last + 1)):
        for c, src_col in zip([12, 13, 14], ["B", "C", "D"]):
            f = f"=MIN({src_col}{rrow},0)^2"
            cell = rr.cell(row=rrow, column=c, value=f)
            cell.number_format = "0.000%"
            style_body(cell, size=8, color="7F7F7F")

    # ---- Drawdown detail (real running-peak method), cols A..J starting a few rows below ----
    dd_title_row = ret_last + 3
    rr.merge_cells(f"A{dd_title_row}:J{dd_title_row}")
    rr[f"A{dd_title_row}"] = "DRAWDOWN DETAIL — running-peak method (real max drawdown, monthly granularity)"
    style_header(rr[f"A{dd_title_row}"], bg=STEEL)
    dd_hdr_row = dd_title_row + 1
    dd_headers = ["Month", "CEG Px", "CEG RunMax", "CEG DD", "VST Px", "VST RunMax", "VST DD", "CCJ Px", "CCJ RunMax", "CCJ DD"]
    for i, h in enumerate(dd_headers):
        c = rr.cell(row=dd_hdr_row, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK, size=8.5)

    dd_first = dd_hdr_row + 1
    tick_cols_md = {"CEG": "B", "VST": "C", "CCJ": "D"}
    dd_col_map = {"CEG": (2, 3, 4), "VST": (5, 6, 7), "CCJ": (8, 9, 10)}
    for i, md_r in enumerate(range(md_sr, md_lr + 1)):
        drow = dd_first + i
        rr.cell(row=drow, column=1, value=f="='Market Data'!A{md_r}")
        style_label(rr.cell(row=drow, column=1), bold=False, size=8.5)
        for t, (px_c, rm_c, dd_c) in dd_col_map.items():
            md_col = tick_cols_md[t]
            px_cell = rr.cell(row=drow, column=px_c, value=f="='Market Data'!{md_col}{md_r}")
            px_cell.number_format = "$#,##0.00"
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
            style_body(px_cell, size=8.5)
            px_letter = get_column_letter(px_c)
            rm_letter = get_column_letter(rm_c)
            if i == 0:
                rm_formula = f"={px_letter}{drow}"
            else:
                rm_formula = f"=MAX({rm_letter}{drow-1},{px_letter}{drow})"
            rm_cell = rr.cell(row=drow, column=rm_c, value=rm_formula)
            rm_cell.number_format = "$#,##0.00"
            style_body(rm_cell, size=8.5, color="7F7F7F")
            dd_formula = f"=({px_letter}{drow}/{rm_letter}{drow})-1"
            dd_cell = rr.cell(row=drow, column=dd_c, value=dd_formula)
            dd_cell.number_format = "0.0%"
            style_body(dd_cell, size=8.5)
        box(rr, f"A{drow}:J{drow}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    dd_last = dd_first + (md_lr - md_sr)
    box(rr, f"A{dd_hdr_row}:J{dd_hdr_row}")

    # ---- Summary stats block ----
    sum_title_row = dd_last + 2
    rr.merge_cells(f"A{sum_title_row}:E{sum_title_row}")
    rr[f"A{sum_title_row}"] = "SUMMARY RISK & RETURN STATISTICS"
    style_header(rr[f"A{sum_title_row}"], bg=STEEL)
    hdr_row = sum_title_row + 1
    for i, h in enumerate(["Metric", "CEG", "VST", "CCJ", "XLE"]):
        c = rr.cell(row=hdr_row, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE if i > 0 else NAVY, color=BLACK if i > 0 else WHITE)

    rf_row = hdr_row + 1
    rr.cell(row=rf_row, column=1, value="Risk-free rate (annual, input)")
    style_label(rr.cell(row=rf_row, column=1), bold=False, italic=True, size=9.5)
    rf_cell = rr.cell(row=rf_row, column=2, value=0.043)
    rf_cell.number_format = "0.0%"
    style_body(rf_cell, size=9.5, bold=True, input_cell=True)
    rf_cell.fill = PatternFill("solid", fgColor=YELLOW_FILL)
    box(rr, f"A{rf_row}:E{rf_row}")

    row = rf_row + 1
    stock_cols = {"CEG": "B", "VST": "C", "CCJ": "D", "XLE": "E"}
    metric_rows = {}

    def metric_row(label, numfmt, formula_fn):
        nonlocal row
        rr.cell(row=row, column=1, value=label)
        style_label(rr.cell(row=row, column=1), bold=False, size=9.5)
        for t, col in stock_cols.items():
            f = formula_fn(t, col)
            cell = rr.cell(row=row, column=["CEG", "VST", "CCJ", "XLE"].index(t) + 2, value=f)
            cell.number_format = numfmt
            style_body(cell, size=9.5)
            cell.alignment = Alignment(horizontal="center")
        box(rr, f"A{row}:E{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
        metric_rows[label] = row
        row += 1

    metric_row("Mean monthly return", "0.00%", lambda t, c: f"=AVERAGE({c}{ret_first}:{c}{ret_last})")
    metric_row("Std dev, monthly", "0.00%", lambda t, c: f"=STDEV({c}{ret_first}:{c}{ret_last})")
    metric_row("Annualized return (CAGR)", "0.0%",
               lambda t, c: f"=('Market Data'!{mdc[t]}{md_lr}/'Market Data'!{mdc[t]}{md_sr})^(12/{md_l
r-md_sr})-1")
    annvol_formula = lambda t, c: f"={c}{metric_rows['Std dev, monthly']}*SQRT(12)"
    metric_row("Annualized volatility", "0.0%", annvol_formula)
    metric_row("Max drawdown (real, running-peak)", "0.0%",
               lambda t, c: (f"=MIN({dd_col_letter(t, 'dd')}{dd_first}:{dd_col_letter(t,'dd')}{dd_last
})"
                             if t != "XLE" else "=\"n/a\""))
    metric_row("Skewness (monthly returns)", "0.00",
               lambda t, c: f"=SKEW({c}{ret_first}:{c}{ret_last})")
    metric_row("Kurtosis (excess, monthly)", "0.00",
               lambda t, c: f"=KURT({c}{ret_first}:{c}{ret_last})")
    metric_row("Historical VaR 95% (monthly)", "0.00%",
               lambda t, c: f"=PERCENTILE({c}{ret_first}:{c}{ret_last},0.05)")
    metric_row("Historical VaR 99% (monthly)", "0.00%",
               lambda t, c: f"=PERCENTILE({c}{ret_first}:{c}{ret_last},0.01)")
    metric_row("Historical CVaR 95% (monthly)", "0.00%",
               lambda t, c: f"=AVERAGEIF({c}{ret_first}:{c}{ret_last},\"<=\"&{c}{metric_rows['Historic
al VaR 95% (monthly)']})")

    downside_cols = {"CEG": "L", "VST": "M", "CCJ": "N"}

    def sortino_fn(t, c):
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
        if t == "XLE":
            return "=\"n/a\""
        dcol = downside_cols[t]
        return f"=({c}{metric_rows['Annualized return (CAGR)']}-$B${rf_row})/(SQRT(AVERAGE({dcol}{ret_first}:{dcol}{ret_last}))*SQRT(12))"

    metric_row("Sortino ratio (annualized, MAR=0)", "0.00", sortino_fn)
    metric_row("Sharpe ratio (annualized)", "0.00",
               lambda t, c: f"=({c}{metric_rows['Annualized return (CAGR)']}-$B${rf_row})/{c}{metric_rows['Annualized volatility']}")

    beta_hardcode = {"CEG": 1.12, "VST": 1.41, "CCJ": 1.01, "XLE": 1.00}
    rr.cell(row=row, column=1, value="Beta (vs. S&P 500, reported/sourced)")
    style_label(rr.cell(row=row, column=1), bold=False, size=9.5)
    for t, col in stock_cols.items():
        cell = rr.cell(row=row, column=["CEG", "VST", "CCJ", "XLE"].index(t) + 2, value=beta_hardcode[t])
        cell.number_format = "0.00"
        style_body(cell, size=9.5, input_cell=True)
        cell.alignment = Alignment(horizontal="center")
    box(rr, f"A{row}:E{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
    metric_rows["Beta (vs. S&P 500, reported/sourced)"] = row
    row += 1

    box(rr, f"A{hdr_row}:E{hdr_row}")

    # ---- Correlation matrix ----
    corr_title_row = row + 1
    rr.merge_cells(f"A{corr_title_row}:E{corr_title_row}")
    rr[f"A{corr_title_row}"] = "CORRELATION MATRIX (monthly returns, incl. benchmark)"
    style_header(rr[f"A{corr_title_row}"], bg=STEEL)
    corr_hdr_row = corr_title_row + 1
    order = ["CEG", "VST", "CCJ", "XLE"]
    rr.cell(row=corr_hdr_row, column=1, value="")
    for i, t in enumerate(order):
        c = rr.cell(row=corr_hdr_row, column=2 + i, value=t)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)
    for i, t1 in enumerate(order):
        rrow = corr_hdr_row + 1 + i
        rr.cell(row=rrow, column=1, value=t1)
        style_label(rr.cell(row=rrow, column=1), bold=True, size=9.5)
        for j, t2 in enumerate(order):
            col1, col2 = stock_cols[t1], stock_cols[t2]
            f = f"=CORREL({col1}{ret_first}:{col1}{ret_last},{col2}{ret_first}:{col2}{ret_last})"
            cell = rr.cell(row=rrow, column=2 + j, value=f)
            cell.number_format = "0.00"
            style_body(cell, size=9.5)
            cell.alignment = Alignment(horizontal="center")
            if t1 == t2:
                cell.fill = PatternFill("solid", fgColor=LIGHT_GRAY)
        box(rr, f"A{rrow}:E{rrow}")
    box(rr, f"A{corr_hdr_row}:E{corr_hdr_row}")

    rr.freeze_panes = "A3"

    return {
        "ret_first": ret_first, "ret_last": ret_last,
        "stock_cols": stock_cols, "metric_rows": metric_rows, "rf_row": rf_row,
        "factor_cols": {"WTI": "G", "HH": "H", "UST10Y": "I", "URANIUM": "J"},
    }


_DD_COLS = {
    "CEG": {"px": "B", "runmax": "C", "dd": "D"},
    "VST": {"px": "E", "runmax": "F", "dd": "G"},
    "CCJ": {"px": "H", "runmax": "I", "dd": "J"},
}


def dd_col_letter(ticker, kind):
    return _DD_COLS[ticker][kind]


# ============================================================================
# 4. BENCHMARK & FACTOR ANALYSIS
# ============================================================================
def build_factor_analysis(wb, rr_info):
    fa = new_sheet(wb, "Benchmark & Factors")
    set_col_widths(fa, [20, 11, 11, 11, 3, 11, 11, 11, 11, 11, 11, 11, 11])
    ret_first, ret_last = rr_info["ret_first"], rr_info["ret_last"]
    sc = rr_info["stock_cols"]
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    mr = rr_info["metric_rows"]
    rf_row = rr_info["rf_row"]
    fc = rr_info["factor_cols"]
    tickers = ["CEG", "VST", "CCJ"]

    fa.merge_cells("A1:M1")
    fa["A1"] = "BENCHMARK & MACRO FACTOR ANALYSIS — single-factor regressions, linked to Returns & Risk"
    style_header(fa["A1"], size=13)
    fa.row_dimensions[1].height = 22

    # ---- helper excess-return columns (for tracking error), cols K,L,M rows 4..23 ----
    fa.cell(row=2, column=11, value="CEG-XLE")
    fa.cell(row=2, column=12, value="VST-XLE")
    fa.cell(row=2, column=13, value="CCJ-XLE")
    for c in (11, 12, 13):
        style_header(fa.cell(row=2, column=c), bg="8EA9C1", size=8.5)
    for rrow in range(ret_first, ret_last + 1):
        for c, t in zip([11, 12, 13], tickers):
            f = f"='Returns & Risk'!{sc[t]}{rrow}-'Returns & Risk'!{sc['XLE']}{rrow}"
            cell = fa.cell(row=rrow, column=c, value=f)
            cell.number_format = "0.00%"
            style_body(cell, size=8, color="7F7F7F")

    # ---- Table A: Market/Benchmark-relative metrics ----
    row = 3
    fa.merge_cells(f"A{row}:I{row}")
    fa[f"A{row}"] = "TABLE A — RELATIVE TO XLE (SECTOR BENCHMARK)"
    style_header(fa[f"A{row}"], bg=STEEL)
    row += 1
    hdrA = ["Ticker", "Beta (XLE)", "R²", "Alpha (ann., Jensen)", "Tracking Error (ann.)",
            "Info Ratio", "Up Capture", "Down Capture"]
    for i, h in enumerate(hdrA):
        c = fa.cell(row=row, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)
    row += 1
    tableA_start = row
    xcol = sc["XLE"]
    for i, t in enumerate(tickers):
        col = sc[t]
        rrow = row + i
        fa.cell(row=rrow, column=1, value=t)
        style_label(fa.cell(row=rrow, column=1), bold=True, size=9.5)
        beta_f = f"=SLOPE('Returns & Risk'!{col}{ret_first}:{col}{ret_last},'Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last})"
        rsq_f = f"=RSQ('Returns & Risk'!{col}{ret_first}:{col}{ret_last},'Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last})"
        beta_cell = fa.cell(row=rrow, column=2, value=beta_f)
        beta_cell.number_format = "0.00"
        style_body(beta_cell, size=9.5)
        rsq_cell = fa.cell(row=rrow, column=3, value=rsq_f)
        rsq_cell.number_format = "0.00"
        style_body(rsq_cell, size=9.5)

        annret_row = mr["Annualized return (CAGR)"]
        alpha_f = (f"='Returns & Risk'!{col}{annret_row}-('Returns & Risk'!$B${rf_row}+B{rrow}*"
                   f"('Returns & Risk'!{xcol}{annret_row}-'Returns & Risk'!$B${rf_row}))")
        alpha_cell = fa.cell(row=rrow, column=4, value=alpha_f)
        alpha_cell.number_format = "0.0%"
        style_body(alpha_cell, size=9.5)

        exc_col = {"CEG": "K", "VST": "L", "CCJ": "M"}[t]
        te_f = f"=STDEV({exc_col}{ret_first}:{exc_col}{ret_last})*SQRT(12)"
        te_cell = fa.cell(row=rrow, column=5, value=te_f)
        te_cell.number_format = "0.0%"
        style_body(te_cell, size=9.5)

        ir_f = f"=('Returns & Risk'!{col}{annret_row}-'Returns & Risk'!{xcol}{annret_row})/E{rrow}"
        ir_cell = fa.cell(row=rrow, column=6, value=ir_f)
        ir_cell.number_format = "0.00"
        style_body(ir_cell, size=9.5)

        up_f = (f"=AVERAGEIF('Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last},\">0\","
                f"'Returns & Risk'!{col}{ret_first}:{col}{ret_last})/"
                f"AVERAGEIF('Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last},\">0\","
                f"'Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last})")
        up_cell = fa.cell(row=rrow, column=7, value=up_f)
        up_cell.number_format = "0.00"
        style_body(up_cell, size=9.5)

        down_f = (f"=AVERAGEIF('Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last},\"<0\","
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
                f"'Returns & Risk'!{col}{ret_first}:{col}{ret_last})/"
                f"AVERAGEIF('Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last},\"<0\","
                f"'Returns & Risk'!{xcol}{ret_first}:{xcol}{ret_last})")
        down_cell = fa.cell(row=rrow, column=8, value=down_f)
        down_cell.number_format = "0.00"
        style_body(down_cell, size=9.5)

        for cc in range(2, 9):
            fa.cell(row=rrow, column=cc).alignment = Alignment(horizontal="center")
        box(fa, f"A{rrow}:H{rrow}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    box(fa, f"A{row-1}:H{row-1}")
    tableA_end = row + len(tickers) - 1
    row = tableA_end + 2

    # ---- Table B: macro factor sensitivities (univariate) ----
    fa.merge_cells(f"A{row}:I{row}")
    fa[f"A{row}"] = "TABLE B — MACRO FACTOR SENSITIVITIES (univariate SLOPE/R², one factor at a time)"
    style_header(fa[f"A{row}"], bg=STEEL)
    row += 1
    hdrB = ["Ticker", "β Oil (WTI)", "R²", "β Nat Gas (HH)", "R²", "β Uranium", "R²", "β Rates (10Y Δ)
", "R²"]
    for i, h in enumerate(hdrB):
        c = fa.cell(row=row, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)
    row += 1
    factor_pairs = [("WTI", fc["WTI"]), ("HH", fc["HH"]), ("URANIUM", fc["URANIUM"]), ("UST10Y", fc["U
ST10Y"])]
    for i, t in enumerate(tickers):
        col = sc[t]
        rrow = row + i
        fa.cell(row=rrow, column=1, value=t)
        style_label(fa.cell(row=rrow, column=1), bold=True, size=9.5)
        out_col = 2
        for fname, fcol in factor_pairs:
            beta_f = f"=SLOPE('Returns & Risk'!{col}{ret_first}:{col}{ret_last},'Returns & Risk'!{fcol
}{ret_first}:{fcol}{ret_last})"
            rsq_f = f"=RSQ('Returns & Risk'!{col}{ret_first}:{col}{ret_last},'Returns & Risk'!{fcol}{r
et_first}:{fcol}{ret_last})"
            bc = fa.cell(row=rrow, column=out_col, value=beta_f)
            bc.number_format = "0.00"
            style_body(bc, size=9.5)
            bc.alignment = Alignment(horizontal="center")
            rc = fa.cell(row=rrow, column=out_col + 1, value=rsq_f)
            rc.number_format = "0.00"
            style_body(rc, size=9.5)
            rc.alignment = Alignment(horizontal="center")
            out_col += 2
        box(fa, f"A{rrow}:I{rrow}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    box(fa, f"A{row-1}:I{row-1}")
    row = row + len(tickers) + 2

    fa.merge_cells(f"A{row}:M{row+4}")
    fa.cell(row=row, column=1, value=(
        "Reading this tab: Beta(XLE) > 1 means the stock amplifies sector moves; Alpha is the annualiz
ed excess "
        "return left over after stripping out that market-beta exposure (CAPM/Jensen's alpha) — a posi
tive alpha "
        "means the stock did better than its sector-beta alone would predict. Information Ratio is alp
ha per unit "
        "of tracking error — the standard buy-side measure of whether active risk is being paid for. U
p/Down "
        "Capture above 1.0 in Up and below 1.0 in Down is the textbook 'asymmetric' profile a long-onl
y holder "
        "wants. Table B betas are univariate (each factor regressed alone) because oil, gas, uranium a
nd rates are "
        "themselves correlated, so a true multi-factor model would need to control for that collineari
ty — treat "
        "these as directional signals of sensitivity, not precise, independent exposures."
    ))
    style_body(fa.cell(row=row, column=1), size=8.5, italic=True, color="595959")
    fa.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
    fa.freeze_panes = "A4"
    return {"tableA_start": tableA_start, "tableA_end": tableA_end}


# ============================================================================
# 5. PORTFOLIO OPTIMIZATION (efficient frontier)
# ============================================================================
def build_optimization(wb, rr_info):
    po = new_sheet(wb, "Portfolio Optimization")
    set_col_widths(po, [12, 12, 12, 13, 13, 11, 3, 10, 10, 10, 10])
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    ret_first, ret_last = rr_info["ret_first"], rr_info["ret_last"]
    sc = rr_info["stock_cols"]
    mr = rr_info["metric_rows"]
    rf_row = rr_info["rf_row"]
    tickers = ["CEG", "VST", "CCJ"]

    po.merge_cells("A1:F1")
    po["A1"] = "PORTFOLIO OPTIMIZATION — Mean-Variance Efficient Frontier (CEG / VST / CCJ)"
    style_header(po["A1"], size=13)
    po.row_dimensions[1].height = 22

    # ---- Monthly covariance matrix (sample, ddof=1, via SUMPRODUCT) ----
    po.merge_cells("A3:D3")
    po["A3"] = "MONTHLY COVARIANCE MATRIX (sample, n-1)"
    style_header(po["A3"], bg=STEEL)
    po.cell(row=4, column=1, value="")
    for i, t in enumerate(tickers):
        c = po.cell(row=4, column=2 + i, value=t)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)
    cov_cells = {}
    n = ret_last - ret_first + 1
    for i, t1 in enumerate(tickers):
        rrow = 5 + i
        po.cell(row=rrow, column=1, value=t1)
        style_label(po.cell(row=rrow, column=1), bold=True, size=9.5)
        for j, t2 in enumerate(tickers):
            col1, col2 = sc[t1], sc[t2]
            rng1 = f"'Returns & Risk'!{col1}{ret_first}:{col1}{ret_last}"
            rng2 = f"'Returns & Risk'!{col2}{ret_first}:{col2}{ret_last}"
            avg1 = f"AVERAGE({rng1})"
            avg2 = f"AVERAGE({rng2})"
            f = f"=SUMPRODUCT(({rng1}-{avg1}),({rng2}-{avg2}))/{n-1}"
            cell = po.cell(row=rrow, column=2 + j, value=f)
            cell.number_format = "0.00000"
            style_body(cell, size=9)
            cell.alignment = Alignment(horizontal="center")
            cov_cells[(t1, t2)] = f"${get_column_letter(2+j)}${rrow}"
        box(po, f"A{rrow}:D{rrow}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    box(po, "A4:D4")

    annret_row = mr["Annualized return (CAGR)"]

    # ---- Weight grid: 10% steps, i+j<=10, wCCJ = 1-wCEG-wVST ----
    grid_title_row = 10
    po.merge_cells(f"A{grid_title_row}:F{grid_title_row}")
    po[f"A{grid_title_row}"] = "EFFICIENT-FRONTIER GRID (10% weight steps) + NAMED PORTFOLIOS"
    style_header(po[f"A{grid_title_row}"], bg=STEEL)
    hdr_row = grid_title_row + 1
    for i, h in enumerate(["w(CEG)", "w(VST)", "w(CCJ)", "Port. Return", "Port. Vol", "Sharpe"]):
        c = po.cell(row=hdr_row, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)

    grid_first = hdr_row + 1
    row = grid_first
    grid_rows = []
    for i in range(0, 11):
        for j in range(0, 11 - i):
            w_ceg, w_vst = i / 10.0, j / 10.0
            w_ccj = round(1 - w_ceg - w_vst, 4)
            po.cell(row=row, column=1, value=w_ceg)
            po.cell(row=row, column=2, value=w_vst)
            po.cell(row=row, column=3, value=w_ccj)
            for c in (1, 2, 3):
                cell = po.cell(row=row, column=c)
                cell.number_format = "0%"
                style_body(cell, size=8.5, input_cell=True)
                cell.alignment = Alignment(horizontal="center")
            ret_f = (f"=A{row}*'Returns & Risk'!{sc['CEG']}{annret_row}+"
                     f"B{row}*'Returns & Risk'!{sc['VST']}{annret_row}+"
                     f"C{row}*'Returns & Risk'!{sc['CCJ']}{annret_row}")
            ret_cell = po.cell(row=row, column=4, value=ret_f)
            ret_cell.number_format = "0.0%"
            style_body(ret_cell, size=8.5)
            vol_f = (f"=SQRT(A{row}^2*{cov_cells[('CEG','CEG')]}*12 + B{row}^2*{cov_cells[('VST','VST'
)]}*12 + "
                     f"C{row}^2*{cov_cells[('CCJ','CCJ')]}*12 + "
                     f"2*A{row}*B{row}*{cov_cells[('CEG','VST')]}*12 + "
                     f"2*A{row}*C{row}*{cov_cells[('CEG','CCJ')]}*12 + "
                     f"2*B{row}*C{row}*{cov_cells[('VST','CCJ')]}*12)")
            vol_cell = po.cell(row=row, column=5, value=vol_f)
            vol_cell.number_format = "0.0%"
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
            style_body(vol_cell, size=8.5)
            sharpe_f = f"=(D{row}-'Returns & Risk'!$B${rf_row})/E{row}"
            sharpe_cell = po.cell(row=row, column=6, value=sharpe_f)
            sharpe_cell.number_format = "0.00"
            style_body(sharpe_cell, size=8.5)
            for cc in (4, 5, 6):
                po.cell(row=row, column=cc).alignment = Alignment(horizontal="center")
            grid_rows.append(row)
            row += 1
    grid_last = row - 1
    box(po, f"A{hdr_row}:F{hdr_row}")
    for r in range(grid_first, grid_last + 1):
        box(po, f"A{r}:F{r}", bg=LIGHT_GRAY if (r - grid_first) % 2 == 0 else None)

    # ---- Named reference portfolios (own rows, same formula structure) ----
    named_title_row = grid_last + 2
    po.merge_cells(f"A{named_title_row}:F{named_title_row}")
    po[f"A{named_title_row}"] = "NAMED REFERENCE PORTFOLIOS"
    style_header(po[f"A{named_title_row}"], bg="595959")
    named_hdr = named_title_row + 1
    for i, h in enumerate(["w(CEG)", "w(VST)", "w(CCJ)", "Port. Return", "Port. Vol", "Sharpe"]):
        c = po.cell(row=named_hdr, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)
    named = [("Equal-Weight (33/33/33)", 1/3, 1/3, 1/3),
             ("Research Reco (40/35/25)", 0.40, 0.35, 0.25)]
    named_first = named_hdr + 1
    for k, (label, w1, w2, w3) in enumerate(named):
        row = named_first + k
        po.cell(row=row, column=1, value=round(w1, 4))
        po.cell(row=row, column=2, value=round(w2, 4))
        po.cell(row=row, column=3, value=round(w3, 4))
        for c in (1, 2, 3):
            cell = po.cell(row=row, column=c)
            cell.number_format = "0%"
            style_body(cell, size=8.5, input_cell=True)
            cell.alignment = Alignment(horizontal="center")
        ret_f = (f"=A{row}*'Returns & Risk'!{sc['CEG']}{annret_row}+"
                 f"B{row}*'Returns & Risk'!{sc['VST']}{annret_row}+"
                 f"C{row}*'Returns & Risk'!{sc['CCJ']}{annret_row}")
        po.cell(row=row, column=4, value=ret_f).number_format = "0.0%"
        style_body(po.cell(row=row, column=4), size=8.5)
        vol_f = (f"=SQRT(A{row}^2*{cov_cells[('CEG','CEG')]}*12 + B{row}^2*{cov_cells[('VST','VST')]}*
12 + "
                 f"C{row}^2*{cov_cells[('CCJ','CCJ')]}*12 + "
                 f"2*A{row}*B{row}*{cov_cells[('CEG','VST')]}*12 + "
                 f"2*A{row}*C{row}*{cov_cells[('CEG','CCJ')]}*12 + "
                 f"2*B{row}*C{row}*{cov_cells[('VST','CCJ')]}*12)")
        po.cell(row=row, column=5, value=vol_f).number_format = "0.0%"
        style_body(po.cell(row=row, column=5), size=8.5)
        sharpe_f = f"=(D{row}-'Returns & Risk'!$B${rf_row})/E{row}"
        po.cell(row=row, column=6, value=sharpe_f).number_format = "0.00"
        style_body(po.cell(row=row, column=6), size=8.5)
        for cc in (4, 5, 6):
            po.cell(row=row, column=cc).alignment = Alignment(horizontal="center")
        box(po, f"A{row}:F{row}")
    # weight labels would overwrite the numeric weight cells, so names go in col H instead
    for k, (label, *_r) in enumerate(named):
        po.cell(row=named_first + k, column=8, value=label)
        style_label(po.cell(row=named_first + k, column=8), bold=True, size=8.5)
    named_last = named_first + len(named) - 1

    # ---- Optimal portfolios identified via MAX/MIN + INDEX/MATCH ----
    opt_title_row = named_last + 2
    po.merge_cells(f"A{opt_title_row}:F{opt_title_row}")
    po[f"A{opt_title_row}"] = "IDENTIFIED OPTIMAL PORTFOLIOS (from the grid above)"
    style_header(po[f"A{opt_title_row}"], bg="1E7145")
    opt_hdr = opt_title_row + 1
    for i, h in enumerate(["Portfolio", "w(CEG)", "w(VST)", "w(CCJ)", "Port. Return", "Port. Vol", "Sh
arpe"]):
        c = po.cell(row=opt_hdr, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK)
    grid_rng = lambda col: f"{col}{grid_first}:{col}{grid_last}"
    max_sharpe_row = opt_hdr + 1
    po.cell(row=max_sharpe_row, column=1, value="Max-Sharpe")
    style_label(po.cell(row=max_sharpe_row, column=1), bold=True, size=9.5)
    sharpe_rng = grid_rng("F")
    match_f = f"MATCH(MAX({sharpe_rng}),{sharpe_rng},0)"
    for ci, col in zip([2, 3, 4, 5, 6, 7], ["A", "B", "C", "D", "E", "F"]):
        f = f"=INDEX({grid_rng(col)},{match_f})"
        cell = po.cell(row=max_sharpe_row, column=ci, value=f)
        cell.number_format = "0%" if col in ("A", "B", "C") else ("0.0%" if col in ("D", "E") else "0.
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
00")
        style_body(cell, size=9.5, bold=True)
        cell.alignment = Alignment(horizontal="center")
    box(po, f"A{max_sharpe_row}:G{max_sharpe_row}", bg=GREEN_FILL)

    min_vol_row = max_sharpe_row + 1
    po.cell(row=min_vol_row, column=1, value="Min-Volatility")
    style_label(po.cell(row=min_vol_row, column=1), bold=True, size=9.5)
    vol_rng = grid_rng("E")
    match_fv = f"MATCH(MIN({vol_rng}),{vol_rng},0)"
    for ci, col in zip([2, 3, 4, 5, 6, 7], ["A", "B", "C", "D", "E", "F"]):
        f = f"=INDEX({grid_rng(col)},{match_fv})"
        cell = po.cell(row=min_vol_row, column=ci, value=f)
        cell.number_format = "0%" if col in ("A", "B", "C") else ("0.0%" if col in ("D", "E") else "0.
00")
        style_body(cell, size=9.5, bold=True)
        cell.alignment = Alignment(horizontal="center")
    box(po, f"A{min_vol_row}:G{min_vol_row}", bg=LIGHT_BLUE)
    box(po, f"A{opt_hdr}:G{opt_hdr}")

    # ---- Charts: efficient frontier scatter + CML ----
    scatter = ScatterChart()
    scatter.title = "Efficient Frontier — Risk vs. Return"
    scatter.x_axis.title = "Annualized Volatility"
    scatter.y_axis.title = "Annualized Return"
    scatter.x_axis.numFmt = "0%"
    scatter.y_axis.numFmt = "0%"
    scatter.x_axis.delete = False
    scatter.y_axis.delete = False
    scatter.height, scatter.width = 11, 20

    xvals = Reference(po, min_col=5, min_row=grid_first, max_row=grid_last)
    yvals = Reference(po, min_col=4, min_row=grid_first, max_row=grid_last)
    frontier_series = Series(yvals, xvals, title="Feasible Set (grid)")
    frontier_series.marker = Marker(symbol="circle", size=5)
    frontier_series.marker.graphicalProperties.solidFill = "A6A6A6"
    frontier_series.marker.graphicalProperties.line.noFill = True
    frontier_series.graphicalProperties.line.noFill = True
    scatter.series.append(frontier_series)

    max_sharpe_x = Reference(po, min_col=6, min_row=max_sharpe_row, max_row=max_sharpe_row)
    max_sharpe_y = Reference(po, min_col=5, min_row=max_sharpe_row, max_row=max_sharpe_row)
    ms_series = Series(max_sharpe_y, max_sharpe_x, title="Max-Sharpe")
    ms_series.marker = Marker(symbol="star", size=14)
    ms_series.marker.graphicalProperties.solidFill = "C9A646"
    ms_series.graphicalProperties.line.noFill = True
    scatter.series.append(ms_series)

    mv_x = Reference(po, min_col=6, min_row=min_vol_row, max_row=min_vol_row)
    mv_y = Reference(po, min_col=5, min_row=min_vol_row, max_row=min_vol_row)
    mv_series = Series(mv_y, mv_x, title="Min-Vol")
    mv_series.marker = Marker(symbol="diamond", size=12)
    mv_series.marker.graphicalProperties.solidFill = "1F4E79"
    mv_series.graphicalProperties.line.noFill = True
    scatter.series.append(mv_series)

    reco_x = Reference(po, min_col=6, min_row=named_first + 1, max_row=named_first + 1)
    reco_y = Reference(po, min_col=5, min_row=named_first + 1, max_row=named_first + 1)
    reco_series = Series(reco_y, reco_x, title="Research Reco (40/35/25)")
    reco_series.marker = Marker(symbol="triangle", size=12)
    reco_series.marker.graphicalProperties.solidFill = "B22222"
    reco_series.graphicalProperties.line.noFill = True
    scatter.series.append(reco_series)

    po.add_chart(scatter, "H10")

    po.freeze_panes = "A12"
    return {"grid_first": grid_first, "grid_last": grid_last,
            "max_sharpe_row": max_sharpe_row, "min_vol_row": min_vol_row,
            "named_first": named_first, "named": named}


# ============================================================================
# 6. MONTE CARLO SIMULATION (pre-computed by src/monte_carlo.py)
# ============================================================================
def build_monte_carlo(wb):
    mc = new_sheet(wb, "Monte Carlo")
    set_col_widths(mc, [14, 10, 10, 10, 10, 10, 10, 10])

    mc.merge_cells("A1:H1")
    mc["A1"] = "MONTE CARLO SIMULATION — 12-Month Portfolio Outcomes (10,000 paths)"
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    style_header(mc["A1"], size=13)
    mc.row_dimensions[1].height = 22

    mc.merge_cells("A2:H2")
    mc["A2"] = ("Pre-computed by src/monte_carlo.py (seed=42, multivariate-normal monthly returns fit to the "
                "20-month sample, 40/35/25 CEG/VST/CCJ weights). Re-run that script to refresh these values — "
                "they are NOT live Excel formulas. See Cover tab for method and limitations.")
    style_body(mc["A2"], size=8.5, italic=True, color="7F7F7F")
    mc["A2"].alignment = Alignment(wrap_text=True, vertical="top")
    mc.row_dimensions[2].height = 28

    out_dir = os.path.join(HERE, "..", "output")
    summary = read_csv_dict(os.path.join(out_dir, "monte_carlo_summary.csv"))[0]
    fan = read_csv_dict(os.path.join(out_dir, "monte_carlo_fan.csv"))
    hist = read_csv_dict(os.path.join(out_dir, "monte_carlo_histogram.csv"))

    # ---- Summary stats ----
    row = 4
    mc.merge_cells(f"A{row}:B{row}")
    mc[f"A{row}"] = "SIMULATION SUMMARY (terminal, 12-month)"
    style_header(mc[f"A{row}"], bg=STEEL)
    row += 1
    label_map = [
        ("Simulations run", summary["n_sims"], "#,##0"),
        ("Mean terminal return", float(summary["mean_terminal_return"]), "0.0%"),
        ("Median terminal return", float(summary["median_terminal_return"]), "0.0%"),
        ("Std dev, terminal return", float(summary["stdev_terminal_return"]), "0.0%"),
        ("Simulated VaR 95% (12-mo)", float(summary["var_95"]), "0.0%"),
        ("Simulated VaR 99% (12-mo)", float(summary["var_99"]), "0.0%"),
        ("Simulated CVaR 95% (12-mo)", float(summary["cvar_95"]), "0.0%"),
        ("Simulated CVaR 99% (12-mo)", float(summary["cvar_99"]), "0.0%"),
        ("P(loss) over 12 months", float(summary["prob_loss"]), "0.0%"),
        ("P(loss > 10%) over 12 months", float(summary["prob_loss_gt_10pct"]), "0.0%"),
        ("P(gain > 25%) over 12 months", float(summary["prob_gain_gt_25pct"]), "0.0%"),
    ]
    for i, (label, val, fmt) in enumerate(label_map):
        r = row + i
        mc.cell(row=r, column=1, value=label)
        style_label(mc.cell(row=r, column=1), bold=False, size=9.5)
        cell = mc.cell(row=r, column=2, value=val)
        cell.number_format = fmt
        style_body(cell, size=9.5, bold=True, input_cell=True)
        box(mc, f"A{r}:B{r}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    box(mc, f"A{row-1}:B{row-1}")
    summary_last = row + len(label_map) - 1

    # ---- Percentile fan table (for chart) ----
    fan_title_row = summary_last + 2
    mc.merge_cells(f"A{fan_title_row}:H{fan_title_row}")
    mc[f"A{fan_title_row}"] = "PERCENTILE FAN — PORTFOLIO INDEX PATH (start = 1.00)"
    style_header(mc[f"A{fan_title_row}"], bg=STEEL)
    fan_hdr = fan_title_row + 1
    fan_cols = ["month", "p5", "p10", "p25", "p50", "p75", "p90", "p95"]
    for i, h in enumerate(fan_cols):
        c = mc.cell(row=fan_hdr, column=1 + i, value=h.upper())
        style_header(c, bg=LIGHT_BLUE, color=BLACK, size=9)
    fan_first = fan_hdr + 1
    for i, r in enumerate(fan):
        rr = fan_first + i
        mc.cell(row=rr, column=1, value=int(float(r["month"])))
        style_body(mc.cell(row=rr, column=1), size=9)
        for j, col in enumerate(fan_cols[1:]):
            cell = mc.cell(row=rr, column=2 + j, value=float(r[col]))
            cell.number_format = "0.00"
            style_body(cell, size=9, input_cell=True)
            cell.alignment = Alignment(horizontal="center")
        box(mc, f"A{rr}:H{rr}", bg=LIGHT_GRAY if i % 2 == 0 else None)
    fan_last = fan_first + len(fan) - 1
    box(mc, f"A{fan_hdr}:H{fan_hdr}")

    fan_chart = LineChart()
    fan_chart.title = "Simulated Portfolio Paths — Percentile Fan (12 months)"
    fan_chart.y_axis.title = "Portfolio Index (start = 1.00)"
    fan_chart.x_axis.title = "Month"
    fan_chart.height, fan_chart.width = 10, 22
    cats = Reference(mc, min_col=1, min_row=fan_first, max_row=fan_last)
    colors = ["C0C0C0", "9DC3E6", "548235", "1F4E79", "548235", "9DC3E6", "C0C0C0"]
    for i, col_idx in enumerate(range(2, 9)):
        data = Reference(mc, min_col=col_idx, min_row=fan_hdr, max_row=fan_last)
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
        fan_chart.add_data(data, titles_from_data=True)
    fan_chart.set_categories(cats)
    for i, s in enumerate(fan_chart.series):
        s.graphicalProperties.line.solidFill = colors[i]
        s.graphicalProperties.line.width = 16000 if i != 3 else 28000
        s.smooth = False
    fan_chart.legend.position = "b"
    mc.add_chart(fan_chart, f"J{fan_title_row}")

    # ---- Histogram ----
    hist_title_row = fan_last + 2
    mc.merge_cells(f"A{hist_title_row}:H{hist_title_row}")
    mc[f"A{hist_title_row}"] = "TERMINAL RETURN DISTRIBUTION (histogram, 30 bins)"
    style_header(mc[f"A{hist_title_row}"], bg=STEEL)
    hist_hdr = hist_title_row + 1
    for i, h in enumerate(["Bin (left edge)", "Frequency"]):
        c = mc.cell(row=hist_hdr, column=1 + i, value=h)
        style_header(c, bg=LIGHT_BLUE, color=BLACK, size=9)
    hist_first = hist_hdr + 1
    for i, r in enumerate(hist):
        rr = hist_first + i
        c1 = mc.cell(row=rr, column=1, value=float(r["bin_left"]))
        c1.number_format = "0.0%"
        style_body(c1, size=8.5, input_cell=True)
        c2 = mc.cell(row=rr, column=2, value=float(r["freq"]))
        c2.number_format = "0.00%"
        style_body(c2, size=8.5, input_cell=True)
    hist_last = hist_first + len(hist) - 1
    box(mc, f"A{hist_hdr}:B{hist_hdr}")

    hist_chart = BarChart()
    hist_chart.type = "col"
    hist_chart.title = "Terminal (12-mo) Return Distribution"
    hist_chart.y_axis.title = "Frequency"
    hist_chart.x_axis.title = "12-mo Portfolio Return"
    hist_chart.height, hist_chart.width = 10, 22
    hdata = Reference(mc, min_col=2, min_row=hist_hdr, max_row=hist_last)
    hcats = Reference(mc, min_col=1, min_row=hist_first, max_row=hist_last)
    hist_chart.add_data(hdata, titles_from_data=True)
    hist_chart.set_categories(hcats)
    hist_chart.series[0].graphicalProperties.solidFill = "2E5A88"
    hist_chart.legend = None
    hist_chart.gapWidth = 10
    mc.add_chart(hist_chart, f"J{hist_title_row}")

    mc.freeze_panes = "A4"
    return {"summary": summary}


# ============================================================================
# 7-9. DEEP DIVES (CEG / VST / CCJ) — fundamentals, technicals, catalysts, bull/bear
# ============================================================================
def _build_deepdive(wb, sheet_name, ticker, name, sector_line, fundamentals, technicals,
                    earnings_hist, catalysts, bull_points, bear_points, sources):
    ws = new_sheet(wb, sheet_name)
    set_col_widths(ws, [26, 16, 16, 4, 16, 30])

    ws.merge_cells("A1:F1")
    ws["A1"] = f"{ticker} — {name}"
    style_header(ws["A1"], size=14)
    ws.row_dimensions[1].height = 24
    ws.merge_cells("A2:F2")
    ws["A2"] = sector_line
    style_body(ws["A2"], size=10.5, italic=True, color="404040")
    ws.row_dimensions[2].height = 16

    row = 4
    ws.merge_cells(f"A{row}:C{row}")
    ws[f"A{row}"] = "KEY FUNDAMENTALS"
    style_header(ws[f"A{row}"], bg=STEEL)
    row += 1
    for label, val in fundamentals:
        ws.cell(row=row, column=1, value=label)
        style_label(ws.cell(row=row, column=1), bold=False, size=9.5)
        ws.cell(row=row, column=2, value=val)
        style_body(ws.cell(row=row, column=2), size=9.5, bold=True)
        box(ws, f"A{row}:C{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
        row += 1
    box(ws, "A4:C4")

    row += 1
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    ws.merge_cells(f"A{row}:C{row}")
    ws[f"A{row}"] = "TECHNICAL PICTURE"
    style_header(ws[f"A{row}"], bg=STEEL)
    row += 1
    tech_start = row
    for label, val in technicals:
        ws.cell(row=row, column=1, value=label)
        style_label(ws.cell(row=row, column=1), bold=False, size=9.5)
        ws.cell(row=row, column=2, value=val)
        style_body(ws.cell(row=row, column=2), size=9.5, bold=True)
        box(ws, f"A{row}:C{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
        row += 1
    box(ws, f"A{tech_start}:C{tech_start}")

    row += 2
    ws.merge_cells(f"A{row}:F{row}")
    ws[f"A{row}"] = "RECENT QUARTERLY EARNINGS HISTORY"
    style_header(ws[f"A{row}"], bg=STEEL)
    row += 1
    headers = ["Quarter", "Report Date", "EPS Est.", "EPS Actual", "Surprise", "Price Reaction"]
    for i, h in enumerate(headers):
        style_header(ws.cell(row=row, column=1 + i, value=h), bg=LIGHT_BLUE, color=BLACK)
    row += 1
    for q in earnings_hist:
        for i, val in enumerate(q):
            c = ws.cell(row=row, column=1 + i, value=val)
            style_body(c, size=9.5)
            c.alignment = Alignment(horizontal="center")
        surprise_cell = ws.cell(row=row, column=5)
        if isinstance(surprise_cell.value, str) and surprise_cell.value.startswith("-"):
            surprise_cell.fill = PatternFill("solid", fgColor=RED_FILL)
        elif isinstance(surprise_cell.value, str) and surprise_cell.value.startswith("+"):
            surprise_cell.fill = PatternFill("solid", fgColor=GREEN_FILL)
        box(ws, f"A{row}:F{row}")
        row += 1

    row += 1
    ws.merge_cells(f"A{row}:F{row}")
    ws[f"A{row}"] = "CATALYST CALENDAR & RECENT EVENTS (chronological)"
    style_header(ws[f"A{row}"], bg=STEEL)
    row += 1
    for date_s, event in catalysts:
        ws.cell(row=row, column=1, value=date_s)
        style_label(ws.cell(row=row, column=1), bold=True, size=9.5)
        ws.merge_cells(f"B{row}:F{row}")
        ws.cell(row=row, column=2, value=event)
        style_body(ws.cell(row=row, column=2), size=9.5)
        ws.cell(row=row, column=2).alignment = Alignment(wrap_text=True, vertical="top")
        ws.row_dimensions[row].height = 28
        box(ws, f"A{row}:F{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
        row += 1

    row += 1
    ws.cell(row=row, column=1, value="BULL CASE")
    style_header(ws.cell(row=row, column=1), bg="375623", color=WHITE)
    ws.merge_cells(f"B{row}:F{row}")
    row += 1
    for pt in bull_points:
        ws.merge_cells(f"A{row}:F{row}")
        ws.cell(row=row, column=1, value=f"✓  {pt}")
        style_body(ws.cell(row=row, column=1), size=9.5)
        ws.cell(row=row, column=1).fill = PatternFill("solid", fgColor=GREEN_FILL)
        ws.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
        ws.row_dimensions[row].height = 26
        row += 1

    row += 1
    ws.cell(row=row, column=1, value="BEAR CASE / RISKS")
    style_header(ws.cell(row=row, column=1), bg="8B0000", color=WHITE)
    ws.merge_cells(f"B{row}:F{row}")
    row += 1
    for pt in bear_points:
        ws.merge_cells(f"A{row}:F{row}")
        ws.cell(row=row, column=1, value=f"⚠  {pt}")
        style_body(ws.cell(row=row, column=1), size=9.5)
        ws.cell(row=row, column=1).fill = PatternFill("solid", fgColor=RED_FILL)
        ws.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
        ws.row_dimensions[row].height = 26
        row += 1

    row += 1
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    ws.merge_cells(f"A{row}:F{row}")
    ws.cell(row=row, column=1, value="Sources: " + sources)
    style_body(ws.cell(row=row, column=1), size=8, color="7F7F7F", italic=True)
    ws.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
    ws.row_dimensions[row].height = 30
    ws.freeze_panes = "A4"


def build_deep_dives(wb):
    _build_deepdive(
        wb, "CEG Deep Dive", "CEG", "Constellation Energy Corporation",
        "Sector: Utilities — Independent Power Producers | Largest U.S. nuclear fleet operator (~31,676 MW) | Baltimore, MD",
        fundamentals=[
            ("Price (2026-10-01)", "$264.58"), ("Market capitalization", "~$92.3B"),
            ("52-week range", "$228.63 — $412.70"), ("P/E ratio (TTM)", "25.5x"),
            ("Forward P/E", "21.4x"), ("EPS (TTM)", "$10.24 — $10.33"),
            ("Revenue (TTM)", "$31.27B (+26.0% YoY)"), ("Dividend yield", "0.64%"),
            ("Beta (5Y monthly)", "1.12"), ("FY2026 EPS guidance (raised)", "$11.50 — $12.50"),
            ("Analyst consensus", "Buy / Strong Buy"), ("Avg. 12-mo price target", "~$379 (BMO, Aug-2026)"),
        ],
        technicals=[
            ("Trend status", "Below 200-day SMA; lower half of 52-wk range"),
            ("Price vs. 52-wk high/low", "-35.9% / +15.7%"),
            ("1-year price return", "-20.3%"),
            ("Key support / resistance", "~$250 support / ~$310–330 resistance"),
        ],
        earnings_hist=[
            ("Q2 2026", "2026-08-06", "$2.28", "$2.55", "+11.7%", "Guidance raised to $11.50-12.50"),
            ("Q1 2026", "2026-05-11", "$2.60", "$2.74", "+5.3%", "-1.3% (1-day)"),
            ("Q4 2025", "2026-02-24", "N/A", "$2.30", "+2.4%", "+6.4% (1-day)"),
            ("Q3 2025", "2025-11-07", "$3.13", "$3.04", "-2.9%", "+2.0% (1-day)"),
        ],
        catalysts=[
            ("2026-08-06", "Q2 2026 results; FY26 guidance raised to $11.50-12.50; 920 MW of new long-term PPAs signed."),
            ("2026 (ongoing)", "FERC/NRC approvals advance Crane Clean Energy Center (former TMI Unit 1) restart for Microsoft."),
            ("2026 (ongoing)", "Calpine acquisition ($16.4B) integration; divesting Brazos Valley + 4.4GW PJM assets to LS Power for DOJ clearance."),
            ("2026-11-09", "Q3 2026 earnings (confirmed). Consensus EPS $3.65–$3.67."),
        ],
        bull_points=[
            "Largest U.S. nuclear fleet = irreplaceable carbon-free baseload exactly as hyperscalers sign 20-yr PPAs for that profile.",
            "Guidance raised twice in 2026 even as the stock sits ~36% below its Oct-2025 high — a valuation reset, not a broken thesis.",
        ],
        bear_points=[
            "Premium multiple (25x+ TTM) for a utility, even after the pullback.",
            "Large, complex Calpine integration with forced divestitures still in progress; PJM market-design intervention risk sector-wide.",
        ],
        sources="SEC EDGAR 8-K, TipRanks, StockAnalysis.com, Yahoo Finance — retrieved 2026-10-01.",
    )
    _build_deepdive(
        wb, "VST Deep Dive", "VST", "Vistra Corp.",
        "Sector: Utilities — Independent Power Producers | Integrated retail + generation (~39GW) | Irving, TX",
        fundamentals=[
            ("Price (2026-10-01)", "$139.05"), ("Market capitalization", "~$46.4B"),
            ("52-week range", "$132.66 — $217.10"), ("P/E ratio (TTM)", "21.5x — 23.8x"),
            ("EPS (TTM)", "$6.00"), ("Revenue (FY2025)", "$17.74B (+3.0% YoY)"),
            ("Dividend yield", "0.65%"), ("Beta (5Y monthly, reported)", "1.41 (cross-provider range 0.4–1.9)"),
            ("FY2026 adj. EBITDA guidance", "$6.8B — $7.6B (reaffirmed)"),
            ("Credit rating", "BBB- (Fitch upgrade, Mar-2026)"), ("Analyst consensus", "Strong Buy"),
            ("Avg. 12-mo price target", "~$217.58 (+54.5% implied)"),
        ],
        technicals=[
            ("Trend status", "Downtrend since Aug-2025 ($217) to ~$139"),
            ("Price vs. 52-wk high/low", "-36.0% / +4.8%"),
            ("1-year price return", "-32.4%"),
            ("Key support / resistance", "~$132–138 support / ~$160–175 resistance"),
        ],
        earnings_hist=[
            ("Q2 2026", "2026-08-07", "N/A", "N/A", "N/A", "YTD adj. EBITDA $3.261B vs $2.589B PY"),
            ("Q1 2026", "2026-05-11", "N/A", "N/A", "Record Q1 EBITDA", "+5% premarket"),
            ("FY2025", "2026-02 (approx.)", "N/A", "$752M net income", "-69.5% YoY", "Weather/deal-related noise"),
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
            ("Q3 2025", "2025-11-06", "N/A", "N/A", "N/A", "-6.1% (1-day)"),
        ],
        catalysts=[
            ("2026-01-12", "20-year, 2.6 GW nuclear PPA with Meta — flagship zero-carbon long-duration contract."),
            ("2026-01-15", "PJM market-intervention headline risk — shares fell ~9% on affordability-regulation reports."),
            ("2026-03", "Fitch upgraded Vistra to investment grade (BBB-)."),
            ("2026 H2 (pending)", "Cogentrix gas portfolio acquisition (5,500 MW) targeted to close; not yet in guidance."),
            ("2026-11-06", "Q3 2026 earnings (confirmed) — first quarter with potential Cogentrix commentary."),
        ],
        bull_points=[
            "Purest merchant-power / direct-hyperscaler-PPA exposure of the three (Meta 2.6GW/20-yr deal).",
            "Investment-grade upgrade lowers financing cost during an active M&A phase; highest analyst conviction of the group (~55% implied upside).",
        ],
        bear_points=[
            "Most exposed of the three to PJM/ERCOT regulatory intervention — already proven with a 9% single-day drop.",
            "Merchant power economics are inherently more volatile than regulated/contracted revenue; Cogentrix deal unclosed.",
        ],
        sources="StockAnalysis.com, TradingView, Robinhood, Morningstar, PR Newswire — retrieved 2026-10-01.",
    )
    _build_deepdive(
        wb, "CCJ Deep Dive", "CCJ", "Cameco Corporation",
        "Sector: Energy — Uranium Mining & Nuclear Fuel Cycle | Uranium, Fuel Services, Westinghouse (49%) | Saskatoon, Canada",
        fundamentals=[
            ("Price (2026-10-01)", "$87.39"), ("Market capitalization", "~$37.8B"),
            ("52-week range", "$77.70 — $135.24"), ("P/E ratio (TTM)", "~147x–152x (thin TTM earnings)"),
            ("EPS (TTM)", "$0.57"), ("Revenue (FY2025)", "$3.48B (+11.0% YoY)"),
            ("Net income (FY2025)", "$589.6M (+243.1% YoY)"), ("Dividend yield", "0.20%"),
            ("Beta (5Y monthly)", "1.01"), ("Analyst consensus", "Buy / Moderate Buy"),
            ("Avg. 12-mo price target", "$127.77 — $148.68 (wide dispersion)"),
        ],
        technicals=[
            ("Trend status", "Round-trip: $78 (Apr-25) → $135 (Oct-25/Jan-26) → ~$87 now"),
            ("Price vs. 52-wk high/low", "-35.4% / +12.5%"),
            ("1-year price return", "-4.2% (highly path-dependent)"),
            ("Key support / resistance", "~$86–90 support / ~$110–123 resistance"),
        ],
        earnings_hist=[
            ("Q2 2026", "2026-07-31", "$0.26", "$0.13", "-50% to -64%", "-2.1% to -3.2% (1-day)"),
            ("Q1 2026", "2026-05-05", "$0.25–$0.36", "$0.22–$0.34", "mixed", "-3.2% on initial miss"),
            ("Q4 2025", "2026-02-13", "$0.32", "$0.33–$0.36", "~0–+28.6%", "N/A"),
            ("Q3 2025", "2025-11-05", "$0.23", "$0.05", "-78.3%", "N/A"),
        ],
        catalysts=[
            ("2026-07-31", "Q2 2026 EPS $0.13 vs $0.26 est. (miss); shares drifted +10.8% higher over the following 33 days."),
            ("2026 (ongoing)", "Westinghouse (49%-owned) IPO speculation — potential embedded-value unlock."),
            ("2026-10-30 (est.)", "Q3 2026 earnings — date not yet formally confirmed (alt. estimate Nov 4)."),
            ("Structural", "Global uranium supply deficit from AI-driven electrification demand + a decade of underinvestment."),
        ],
        bull_points=[
            "Purest upstream fuel-cycle exposure of the three; FY2025 net income +243% YoY shows real operating leverage emerging.",
            "Westinghouse stake is a legitimate, not-fully-priced-in embedded-value catalyst.",
        ],
        bear_points=[
            "Highest volatility of the three and a demanding valuation (147x+ TTM P/E) with little room for further misses.",
            "Two of the last four quarters were significant EPS misses — a recurring pattern, not a one-off; next earnings date unconfirmed.",
        ],
        sources="MarketChameleon, StockAnalysis.com, MarketBeat, Yahoo Finance — retrieved 2026-10-01.",
    )


# ============================================================================
# 10. CATALYST CALENDAR
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
# ============================================================================
def build_catalyst_calendar(wb):
    cal = new_sheet(wb, "Catalyst Calendar")
    set_col_widths(cal, [13, 9, 38, 14, 10])

    cal.merge_cells("A1:E1")
    cal["A1"] = "CONSOLIDATED CATALYST CALENDAR — Oct 2026 through Q1 2027"
    style_header(cal["A1"], size=13)
    cal.row_dimensions[1].height = 22

    headers = ["Date", "Ticker", "Event", "Type", "Confirmed?"]
    for i, h in enumerate(headers):
        style_header(cal.cell(row=2, column=1 + i, value=h), bg=STEEL)

    events = [
        ("2026-10-01", "ALL", "Report-date valuation snapshot; all fundamentals in this workbook as-of this date.", "Reference", "N/A"),
        ("2026-10-30", "CCJ", "Q3 2026 earnings (estimated — not formally confirmed; alt. tracker estimate Nov 4).", "Earnings", "Estimated"),
        ("2026-11-06", "VST", "Q3 2026 earnings. First full quarter with potential Cogentrix commentary.", "Earnings", "Confirmed"),
        ("2026-11-09", "CEG", "Q3 2026 earnings. Consensus EPS $3.65–$3.67.", "Earnings", "Confirmed"),
        ("2026 H2 (TBD)", "VST", "Cogentrix gas portfolio (5,500 MW) acquisition targeted to close.", "M&A / Closing", "Pending"),
        ("2026 (ongoing)", "CEG", "Crane Clean Energy Center restart progress (FERC/NRC approvals granted).", "Regulatory", "In progress"),
        ("2026 (ongoing)", "CEG", "Calpine integration; Brazos Valley + 4.4GW PJM divestiture to LS Power.", "M&A / Regulatory", "In progress"),
        ("2026 (ongoing)", "CCJ", "Westinghouse IPO speculation — potential embedded-value unlock.", "Corporate action", "Speculative"),
        ("2026 (ongoing)", "ALL", "PJM/ERCOT affordability intervention risk — VST most exposed, CEG moderate, CCJ least.", "Regulatory risk", "Monitoring"),
        ("2026 (ongoing)", "CCJ", "Global uranium structural deficit narrative vs. supply growth.", "Macro / Structural", "Ongoing"),
        ("2027 Q1 (est.)", "ALL", "Q4 2026 earnings across all three — first full read on FY2026 guidance delivery and FY2027 initial guides.", "Earnings", "Estimated"),
    ]
    row = 3
    for date_s, ticker, event, etype, conf in events:
        cal.cell(row=row, column=1, value=date_s)
        style_label(cal.cell(row=row, column=1), bold=True, size=9)
        cal.cell(row=row, column=2, value=ticker)
        style_body(cal.cell(row=row, column=2), size=9, bold=True)
        cal.cell(row=row, column=2).alignment = Alignment(horizontal="center")
        cal.cell(row=row, column=3, value=event)
        style_body(cal.cell(row=row, column=3), size=9)
        cal.cell(row=row, column=3).alignment = Alignment(wrap_text=True, vertical="top")
        cal.cell(row=row, column=4, value=etype)
        style_body(cal.cell(row=row, column=4), size=9, italic=True)
        cal.cell(row=row, column=4).alignment = Alignment(horizontal="center", wrap_text=True)
        conf_cell = cal.cell(row=row, column=5, value=conf)
        style_body(conf_cell, size=9, bold=True)
        conf_cell.alignment = Alignment(horizontal="center")
        fill = {"Confirmed": GREEN_FILL, "Estimated": YELLOW_FILL, "Pending": YELLOW_FILL,
                "Monitoring": YELLOW_FILL, "Speculative": RED_FILL, "N/A": None, "In progress": YELLOW_FILL,
                "Ongoing": YELLOW_FILL}.get(conf)
        if fill:
            conf_cell.fill = PatternFill("solid", fgColor=fill)
        cal.row_dimensions[row].height = 30
        box(cal, f"A{row}:E{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
        row += 1
    box(cal, "A2:E2")
    cal.freeze_panes = "A3"


# ============================================================================
# 11. RECOMMENDATION — optimizer + simulation-informed
# ============================================================================
def build_recommendation(wb, rr_info, po_info, mc_info):
    rec = new_sheet(wb, "Recommendation")
    set_col_widths(rec, [24, 14, 14, 14, 4, 30])

    rec.merge_cells("A1:D1")
    rec["A1"] = "PORTFOLIO RECOMMENDATION — Optimizer & Simulation-Informed"
    style_header(rec["A1"], size=13)
    rec.row_dimensions[1].height = 22

    sc = rr_info["stock_cols"]
    mr = rr_info["metric_rows"]
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
1351
1352      row = 3
1353      rec.merge_cells(f"A{row}:D{row}")
1354      rec[f"A{row}"] = "RISK / REWARD SNAPSHOT (live-linked to Returns & Risk)"
1355      style_header(rec[f"A{row}"], bg=STEEL)
1356      row += 1
1357      for i, h in enumerate(["Metric", "CEG", "VST", "CCJ"]):
1358          style_header(rec.cell(row=row, column=1 + i, value=h), bg=LIGHT_BLUE if i else NAVY, color=BLA
      CK if i else WHITE)
1359      row += 1
1360      metrics_to_show = [
1361          ("Annualized return", "Annualized return (CAGR)", "0.0%"),
1362          ("Annualized volatility", "Annualized volatility", "0.0%"),
1363          ("Sharpe ratio", "Sharpe ratio (annualized)", "0.00"),
1364          ("Sortino ratio", "Sortino ratio (annualized, MAR=0)", "0.00"),
1365          ("Max drawdown (real)", "Max drawdown (real, running-peak)", "0.0%"),
1366          ("Historical VaR 95% (monthly)", "Historical VaR 95% (monthly)", "0.00%"),
1367          ("Historical CVaR 95% (monthly)", "Historical CVaR 95% (monthly)", "0.00%"),
1368      ]
1369      for label, src_label, fmt in metrics_to_show:
1370          rec.cell(row=row, column=1, value=label)
1371          style_label(rec.cell(row=row, column=1), bold=False, size=9.5)
1372          src_row = mr[src_label]
1373          for i, t in enumerate(["CEG", "VST", "CCJ"]):
1374              f = f"='Returns & Risk'!{sc[t]}{src_row}"
1375              cell = rec.cell(row=row, column=2 + i, value=f)
1376              cell.number_format = fmt
1377              style_body(cell, size=9.5)
1378              cell.alignment = Alignment(horizontal="center")
1379          box(rec, f"A{row}:D{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
1380          row += 1
1381      box(rec, f"A{row-len(metrics_to_show)-1}:D{row-len(metrics_to_show)-1}")
1382      row += 1
1383
1384      # ---- Optimizer output ----
1385      rec.merge_cells(f"A{row}:D{row}")
1386      rec[f"A{row}"] = "OPTIMIZER OUTPUT (live-linked to Portfolio Optimization)"
1387      style_header(rec[f"A{row}"], bg=STEEL)
1388      row += 1
1389      for i, h in enumerate(["Portfolio", "w(CEG)", "w(VST)", "w(CCJ)"]):
1390          style_header(rec.cell(row=row, column=1 + i, value=h), bg=LIGHT_BLUE if i else NAVY, color=BLA
      CK if i else WHITE)
1391      row += 1
1392      # The "identified optimal" rows hold a label in column A and weights in B:D;
1393      # the named reference portfolios hold weights directly in A:C.
1394      opt_rows = [("Max-Sharpe (optimizer)", po_info["max_sharpe_row"], ["B", "C", "D"]),
1395                  ("Min-Volatility (optimizer)", po_info["min_vol_row"], ["B", "C", "D"]),
1396                  ("Research Recommendation", po_info["named_first"] + 1, ["A", "B", "C"])]
1397      for label, src_row, weight_cols in opt_rows:
1398          rec.cell(row=row, column=1, value=label)
1399          style_label(rec.cell(row=row, column=1), bold=True, size=9.5)
1400          for i, col in enumerate(weight_cols):
1401              f = f"='Portfolio Optimization'!{col}{src_row}"
1402              cell = rec.cell(row=row, column=2 + i, value=f)
1403              cell.number_format = "0%"
1404              style_body(cell, size=9.5)
1405              cell.alignment = Alignment(horizontal="center")
1406          box(rec, f"A{row}:D{row}", bg=LIGHT_GRAY if row % 2 == 0 else None)
1407          row += 1
1408      box(rec, f"A{row-len(opt_rows)-1}:D{row-len(opt_rows)-1}")
1409      row += 1
1410
1411      rec.merge_cells(f"A{row}:D{row+3}")
1412      rec.cell(row=row, column=1, value=(
1413          "Why we don't just take the optimizer's weights: mean-variance optimization on a 20-observatio
      n sample is "
1414          "notoriously unstable — it tends to pile weight into whichever asset happened to have the best
       trailing "
1415          "Sharpe, which is as much noise as signal at this sample size. The Research Recommendation (40
      /35/25) "
1416          "deliberately keeps CCJ's weight below both the optimizer's and an equal-weight split, specifi
      cally because "
1417          "its earnings unpredictability and valuation are a real, separate risk the optimizer's covaria
      nce matrix "
1418          "does not capture. Compare all three rows above before sizing a real position."
1419      ))
1420      style_body(rec.cell(row=row, column=1), size=8.5, italic=True, color="595959")
1421      rec.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
1422      row += 5
1423
1424      # ---- Monte Carlo informed scenario framing ----
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    rec.merge_cells(f"A{row}:D{row}")
    rec[f"A{row}"] = "12-MONTH SCENARIO FRAMING (from Monte Carlo simulation, Research Reco weights)"
    style_header(rec[f"A{row}"], bg=STEEL)
    row += 1
    s = mc_info["summary"]
    scenario_rows = [
        ("Bear (5th percentile)", float(s["p5_terminal_return"])),
        ("Below-average (25th pct.)", float(s["p25_terminal_return"])),
        ("Base / median (50th pct.)", float(s["p50_terminal_return"])),
        ("Above-average (75th pct.)", float(s["p75_terminal_return"])),
        ("Bull (95th percentile)", float(s["p95_terminal_return"])),
    ]
    for i, h in enumerate(["Scenario", "Simulated 12-mo return"]):
        style_header(rec.cell(row=row, column=1 + i, value=h), bg=LIGHT_BLUE if i else NAVY, color=BLACK if i else WHITE)
    row += 1
    for label, val in scenario_rows:
        rec.cell(row=row, column=1, value=label)
        style_label(rec.cell(row=row, column=1), bold=False, size=9.5)
        cell = rec.cell(row=row, column=2, value=val)
        cell.number_format = "+0.0%;-0.0%"
        style_body(cell, size=9.5, bold=True, input_cell=True)
        if val < 0:
            cell.fill = PatternFill("solid", fgColor=RED_FILL)
        elif val > 0.25:
            cell.fill = PatternFill("solid", fgColor=GREEN_FILL)
        else:
            cell.fill = PatternFill("solid", fgColor=YELLOW_FILL)
        box(rec, f"A{row}:B{row}")
        row += 1
    row += 1
    rec.merge_cells(f"A{row}:D{row}")
    rec.cell(row=row, column=1, value=(
        f"Distribution-level risk: P(loss) = {float(s['prob_loss']):.1%} over 12 months; "
        f"P(loss > 10%) = {float(s['prob_loss_gt_10pct']):.1%}; simulated 95% VaR = {float(s['var_95']):.1%}, "
        f"95% CVaR = {float(s['cvar_95']):.1%} — i.e., in the worst 5% of simulated outcomes, the average "
        f"loss is roughly {float(s['cvar_95']):.0%}. This is the honest tail-risk picture that a single 'bear case' "
        f"price target does not convey."
    ))
    style_body(rec.cell(row=row, column=1), size=9, italic=True)
    rec.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
    rec.row_dimensions[row].height = 48
    row += 3

    # ---- Bottom line ----
    rec.merge_cells(f"A{row}:D{row}")
    rec[f"A{row}"] = "BOTTOM LINE"
    style_header(rec[f"A{row}"], bg="1E7145")
    row += 1
    rec.merge_cells(f"A{row}:D{row+5}")
    rec.cell(row=row, column=1, value=(
        "All three names screen Buy-rated with meaningful upside to analyst targets after a broad 2025/2026 sector "
        "de-rating. Benchmarked against XLE, CEG and VST's betas and information ratios (Benchmark & Factors tab) "
        "show how much of their recent weakness is sector-wide vs. idiosyncratic; the factor table shows real, if "
        "imprecise, sensitivity to oil, gas, uranium and rates. The optimizer confirms a barbell shape is reasonable "
        "but, on this short sample, would overweight whichever name had the best trailing luck — a classic "
        "mean-variance pitfall — which is why the Research Recommendation deliberately underweights the "
        "optimizer's top pick relative to its covariance-implied weight. The Monte Carlo simulation replaces a "
        "single eyeballed bear/base/bull price with an actual distribution: a ~44% chance of loss over 12 months "
        "and a 95% CVaR near -54% at these weights is a real, quantified tail risk, not a footnote. This is a "
        "research framework for further diligence, not a trade instruction; position sizing, entry timing, and "
        "risk limits should be set per the investor's own mandate and verified against live market data."
    ))
    style_body(rec.cell(row=row, column=1), size=10)
    rec.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
    row += 7

```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.

```
    rec.merge_cells(f"A{row}:D{row}")
    rec.cell(row=row, column=1, value=(
        "DISCLAIMER: All scenario ranges, optimizer output, and Monte Carlo statistics are illustrativ
e, "
        "author-generated analysis for portfolio-construction discussion only — not price targets, "
        "recommendations, or investment advice. This is not a research report of a registered broker-d
ealer or "
        "investment adviser. See the Cover tab for full methodology and limitations. Consult a license
d financial "
        "advisor and verify all data independently before making any investment decision."
    ))
    style_body(rec.cell(row=row, column=1), size=8, color="7F7F7F", italic=True)
    rec.cell(row=row, column=1).alignment = Alignment(wrap_text=True, vertical="top")
    rec.row_dimensions[row].height = 48
    rec.freeze_panes = "A2"


# ============================================================================
# MAIN
# ============================================================================
def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    if os.path.exists(OUT_PATH):
        wb = openpyxl.Workbook()  # always build fresh for reproducibility
    else:
        wb = openpyxl.Workbook()

    build_cover(wb)
    md_rows = build_market_data(wb)
    rr_info = build_returns_risk(wb, md_rows)
    fa_info = build_factor_analysis(wb, rr_info)
    po_info = build_optimization(wb, rr_info)
    mc_info = build_monte_carlo(wb)
    build_deep_dives(wb)
    build_catalyst_calendar(wb)
    build_recommendation(wb, rr_info, po_info, mc_info)

    # order sheets sensibly
    order = ["Cover", "Market Data", "Returns & Risk", "Benchmark & Factors",
             "Portfolio Optimization", "Monte Carlo", "CEG Deep Dive", "VST Deep Dive",
             "CCJ Deep Dive", "Catalyst Calendar", "Recommendation"]
    wb._sheets = [wb[name] for name in order]
    for name in order:
        wb[name].sheet_state = "visible"

    wb.save(OUT_PATH)
    print("Workbook built:", OUT_PATH)
    print("Sheets:", wb.sheetnames)


if __name__ == "__main__":
    main()
```

Read-only copy · © 2026 Christos Katsampas · Copying, editing or redistribution without written permission is prohibited.